Pipeline Interaction: POS Tags as Features for NER

Egor Chernobrovkin
Innopolis University

Project Link

Motivation

POS tagging and NER are closely related token-level NLP tasks. POS tags may help NER by providing syntactic information about proper nouns, local context, and entity boundaries. However, in realistic pipelines POS tags are predicted automatically and may contain errors.

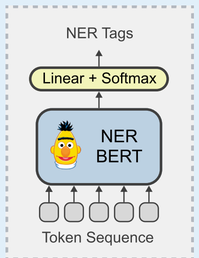
This project studies whether POS features still improve NER under noisy conditions, or whether their benefit is reduced by error propagation.

Experimental Setup

Dataset

Set type	Sentences	Tokens	LOC	MISC	ORG	PER
Training set	14,041	203,621	7,140	3,438	6,321	6,600
Validation set	3,250	51,362	1,837	922	1,341	1,842
Test set	3,453	46,435	1,668	702	1,661	1,617

- CoNLL-2003 (English newswire)
- 47 POS labels and 9 NER labels in one corpus
- **NER entity types:** PER, ORG, LOC, MISC



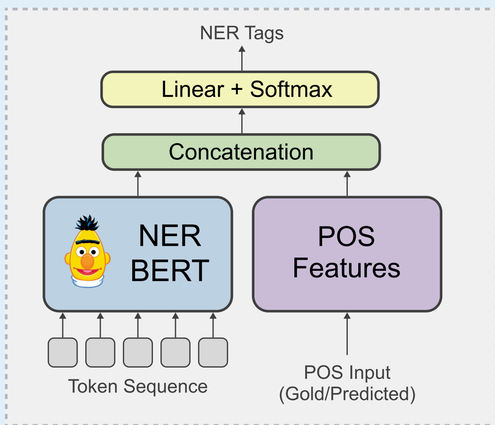
NER Baseline

Experiments

Backbone Model: *bert-base-cased*

1. **NER Baseline**
   BERT token classifier without POS input
2. **NER + Gold POS**
   BERT token classifier with gold POS features
3. **NER + Predicted POS**
   BERT token classifier with predicted POS features

POS Feature Injection for NER



Encoder-independent fusion: POS is added after the encoder, before the final classifier

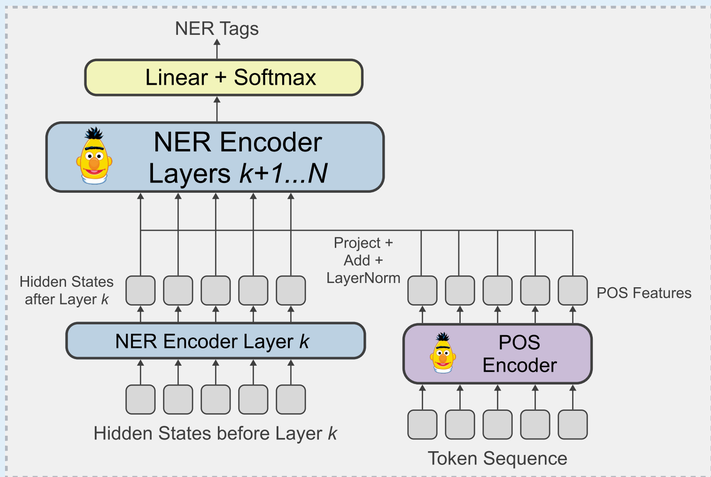
- POS features are concatenated with final NER hidden states;
- The BERT encoder remains unchanged;
- POS is used only by the final classifier (linear + softmax).

Predicted POS setup

Issue:
Predicted POS for NER train can leak if generated by a POS model trained on the same examples.

Solution:
Use 5-fold out-of-fold POS predictions for train; use a full-train POS model for validation and test.

- POS embeddings are injected into NER hidden states at layer k;
- The POS signal is projected to the BERT hidden size;
- The remaining encoder layers process POS-aware hidden states;
- Early encoder fusion injects POS before the first encoder layer;
- Late encoder fusion injects POS near the final encoder layers.



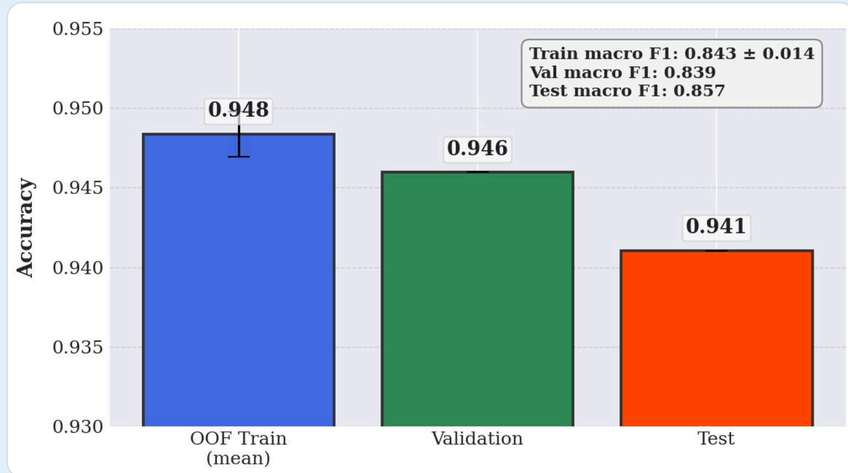
Encoder-level fusion: POS embeddings are injected after layer k of the NER encoder

Comparison of Injection Strategies

Fusion strategy	Injection point	Test Accuracy	Test Macro F1
Encoder-independent fusion	after encoder	0.980	0.879
Early encoder fusion	k = 0	0.966	0.815
Late encoder fusion	k = N − 2	0.980	0.878

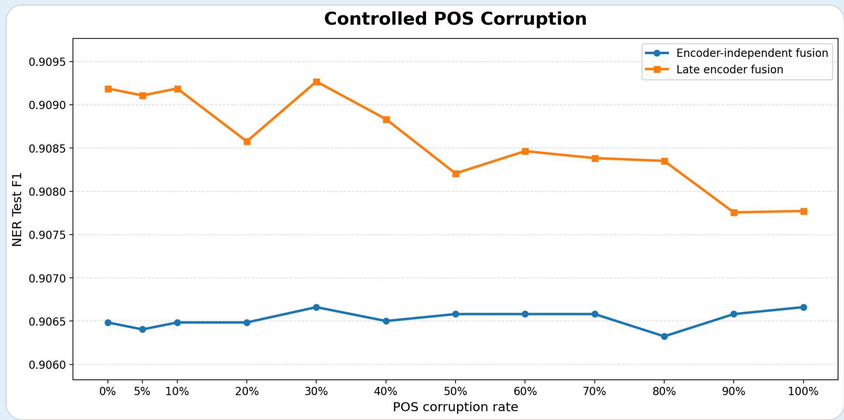
Encoder-independent fusion performed best among POS-aware variants. Early encoder fusion strongly degraded performance, while late encoder fusion was more stable but still did not outperform the simpler encoder-independent setup.

POS Tagging Performance



- **Accuracy = 94.1%**, i.e. errors on about 5.9% of tokens;
- **Macro F1 = 0.857**, showing lower performance on less frequent POS tags.

Error Propagation Analysis



- POS tags were corrupted according to the POS tagger's confusion matrix;
- Even 100% corrupted POS tags reduced F1 by only ≈0.0014.

Results

Model	Test Precision	Test Recall	Test F1	Test Macro F1
Baseline NER	**0.9070 ± 0.0035**	**0.9101 ± 0.0008**	**0.9085 ± 0.0017**	**0.8801 ± 0.0004**
NER + Gold POS	0.9047 ± 0.0011	0.9105 ± 0.0003	0.9076 ± 0.0005	0.8787 ± 0.0018
NER + Predicted POS	0.9050 ± 0.0040	0.9105 ± 0.0019	0.9077 ± 0.0029	0.8792 ± 0.0009

Across three random seeds, neither gold POS nor predicted POS features produced an improvement over the BERT baseline. The differences in test F1 were within the range of seed variation. This suggests that, in our setup, explicit POS information is redundant for BERT-based NER.

Takeaways

- Explicit POS features **did not improve** over the BERT-based NER baseline;
- Predicted POS achieved **similar NER performance** to gold POS;
- Controlled corruption revealed **weak POS-to-NER error propagation**;
- BERT representations **already encode** most useful syntactic information for CoNLL-2003 NER.